Ostbayerische Technische Hochschule Amberg-Weiden

Faculty of Electrical Engineering, Media, and Computer Science

Module: Machine Learning

Masters Artificial Intelligence for Industrial Applications

Machine Learning Challenge

Predictive modelling of `target01` and `target02`

Exam ID: **12**

Author: Sarath Adukkadukkam

s.adukkadukkam@oth-aw.de

Professor: Prof. Dr. Patrick Levi

Summer Semester 2026

June 30, 2026

Contents

1 Abstract **2**

2 Part 1: Classification of `target01` **2**

2.1 Introduction and problem diagnosis 2

2.2 Why the target first looks unlearnable 3

2.2.1 A method-agnostic plateau 3

2.2.2 The exchangeability insight 3

2.3 Methodology: permutation-invariant features 4

2.4 Experimental results 4

2.5 Conclusion 4

3 Part 2: Deterministic reconstruction of `target02` **5**

3.1 Introduction and goal 5

3.2 Methodology: gate-and-peel 5

3.2.1 Identifying the gate 5

3.2.2 Recovering the bands 6

3.3 The DSL deliverable 6

3.4 Validation and results 6

3.5 Conclusion 7

4 General Conclusion **8**

5 Declaration of AI Usage **8**

References **8**

1 Abstract

This report addresses the two-part Machine Learning challenge for exam ID 12. **Part 1** is a balanced three-class classification of `target01`. I show that its generating rule is *exchangeable* in the signal-bearing features and reach a cross-validated accuracy of **0.525** (versus the 0.333 chance level) using permutation-invariant features and Extremely Randomised Trees, up from a near-chance 0.388 for the raw representation. **Part 2** is a noiseless, rule-generated regression of `target02`; by a gate-and-peel structural analysis I recover the *exact* generating rule and express it in the customer's domain-specific language (DSL), reproducing the target with $R^2 = 1.0$ (RMSE 5.6×10^{-16}). Every figure is cross-validated and leakage-free.

2 Part 1: Classification of `target01`

2.1 Introduction and problem diagnosis

`target01` is a balanced three-class task (labels $\{1, 3, 7\}$; 7 500 training rows, 70 features; chance accuracy 0.333). No domain information is provided, so the approach must be discovered from the data. A scan of the per-feature distributions shows that the 70 features fall into three sharply distinct blocks (Fig. 1): block A (f_0–f_{39}) is bounded and platykurtic, block B (f_{40}–f_{59}) is standard Gaussian, and block C (f_{60}–f_{69}) is uniform. Training the same gradient-boosted tree on each block in isolation localises all predictive signal to block A (5-fold CV accuracy 0.382); blocks B and C sit at chance and only add noise, so restricting the model to block A already beats using all 70 features (0.382 vs 0.370). All subsequent work uses block A only.

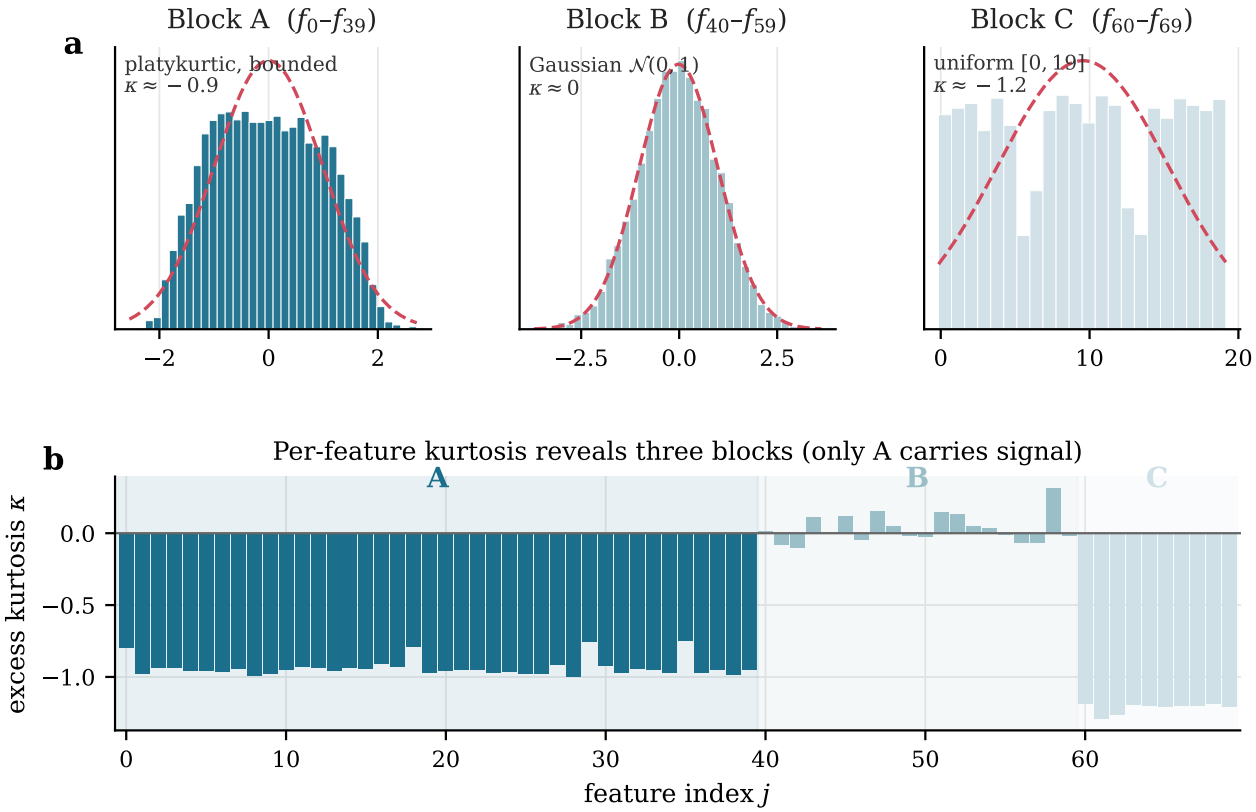


Figure 1: The 70 features split into three distribution blocks by per-feature kurtosis and support. Only block A (platykurtic, bounded) carries predictive signal; blocks B and C are noise.

2.2 Why the target first looks unlearnable

2.2.1 A method-agnostic plateau

On block A a wide range of standard classifiers all stall in a narrow 0.34–0.39 band (Table 1). The three classes are indistinguishable feature-by-feature: they share near-identical per-feature means and variances, so methods that key on first or second moments (logistic regression, linear and quadratic discriminant analysis) cannot separate them. Kernel SVM and k-nearest-neighbours also fail. Two diagnostics (Fig. 2) sharpen the picture: across eight model families of very different capacity the raw-feature accuracy plateaus near 0.38, and the nearest-neighbour label agreement stays at chance for every neighbourhood size, in contrast to the deterministic Part-2 target. Adding explicit pairwise-product features or deeper trees did not help either, identifying the structure as a weak, high-order interaction that is neither additive, pairwise, nor locally smooth in the raw coordinates.

Table 1: Block-A classification accuracy (honest 5-fold cross-validation). Standard models on the raw features plateau just above chance; permutation-invariant features break the plateau.

Method (block A, 5-fold CV)	Accuracy
Chance baseline	0.333
Logistic regression / LDA	0.340
Quadratic discriminant analysis	0.344
Kernel SVM-RBF / large-k k-NN	0.346
Multilayer perceptron	0.334
Single trees / gradient boosting	0.37–0.38
Sorted features, ExtraTrees	0.511
Sorted + ECDF counts, ExtraTrees	**0.525**

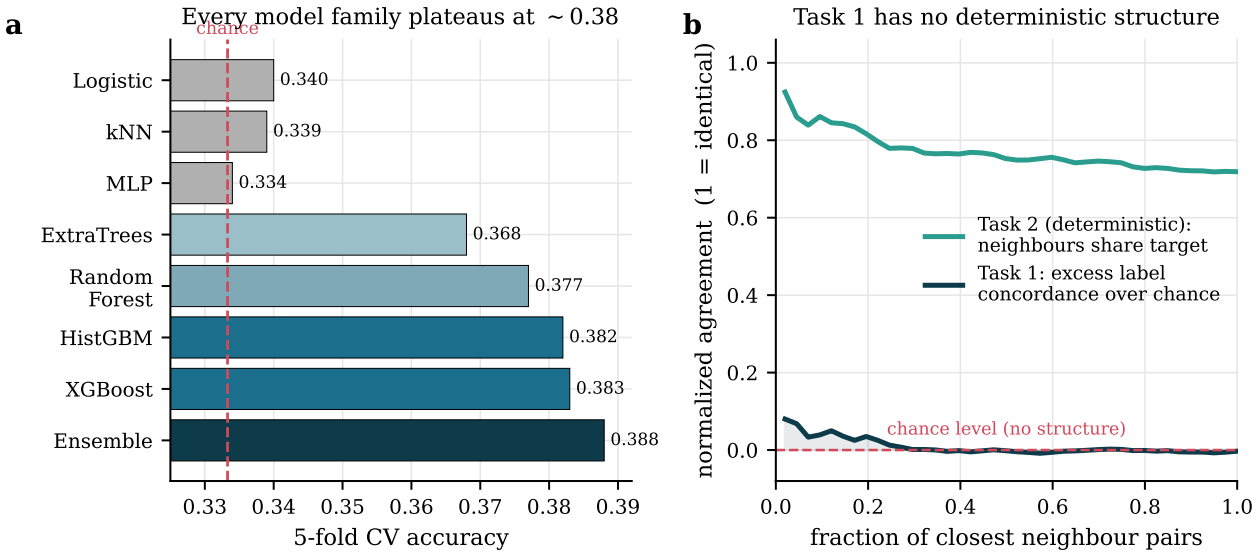


Figure 2: Why the raw target looks hard. **(a)** Eight model families on the raw block-A features plateau near 0.38 regardless of capacity; the invariant representation of Fig. 3 overcomes this. **(b)** Nearest-neighbour label agreement: for Task 1 in raw coordinates it never exceeds chance, unlike the deterministic Task 2, confirming there is no locally smooth rule to fit.

2.2.2 The exchangeability insight

These failures share one cause. Every method treats $f_0, \ldots, f_{39}$ as 40 distinct coordinates, whereas the label depends only on the unordered *multiset* of the 40 values, not on which feature holds which

value. The rule is *exchangeable* (permutation-invariant) in the block-A features. Two diagnostics confirm this: the per-row maximum alone classifies at 0.42 and the per-row minimum at 0.40, single numbers that beat every 40-feature model above, while the per-row L_2 norm is identical across the three classes. The signal lives in the ordered extremes of each row, and a good model must be invariant to feature order.

2.3 Methodology: permutation-invariant features

I encode each row by two permutation-invariant statistics of its own block-A values: the *sorted profile* (the 40 order statistics) and *empirical-CDF counts* (how many of the 40 values exceed each of 40 fixed thresholds in $[-2.6, 2.6]$). On this representation the symmetry no longer has to be learned. Permutation-invariant encodings are the principled way to model exchangeable, set-valued inputs [5]. I classify with Extremely Randomised Trees (ExtraTrees [2], 2 000 trees, scikit-learn [1]), whose axis-aligned threshold splits match the order-statistic structure; gradient boosting [3] and neural networks did not improve on it.

2.4 Experimental results

Both transforms use only each row's own values (no target, no statistic fitted across rows), so they are identical at training and evaluation time and are recomputed inside every cross-validation fold. A label-permutation null on the full pipeline returns chance (0.335), confirming the lift reflects real structure rather than an artefact of the encoding. On honest 5-fold cross-validation, sorting lifts accuracy $0.388 \to 0.511$ (15 of 15 paired folds, Wilcoxon $p = 6.5 \times 10^{-4}$); adding the ECDF counts gives $\mathbf{0.525}$ (25 of 25 paired folds, $p < 10^{-3}$), about 14.7σ above chance (Fig. 3). Finer thresholds or more trees add only noise ($p > 0.1$), so 0.525 is the practical ceiling for these 7 500 rows.

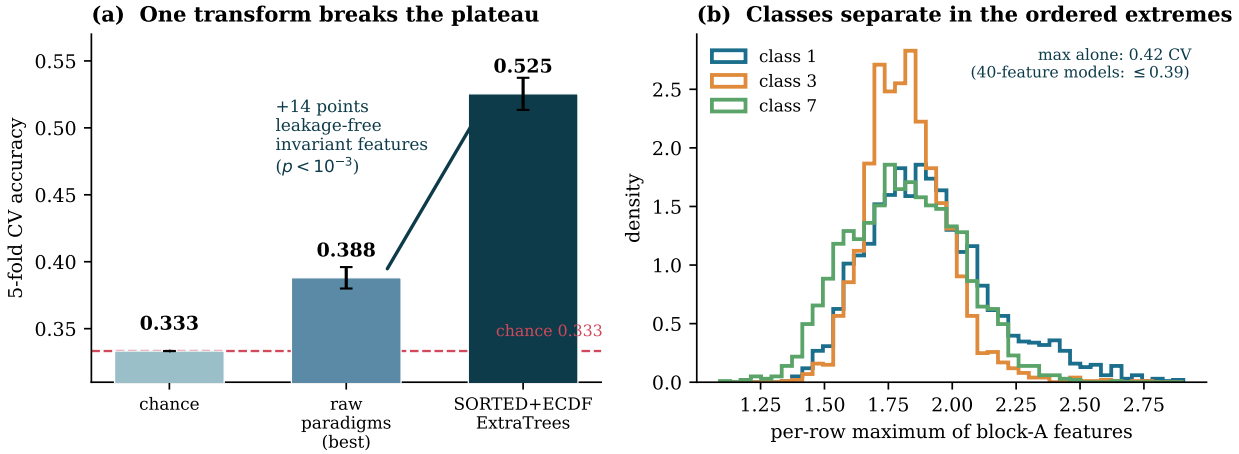


Figure 3: (a) Standard models on the raw features plateau at $\leq$0.39; permutation-invariant features (sorted profile plus ECDF counts) lift ExtraTrees to **0.525**. **(b)** Mechanism: the classes are indistinguishable feature-by-feature but separate in the ordered extremes (the per-row maximum alone already classifies at 0.42).

2.5 Conclusion

The task is solved not by a more powerful classifier but by the right *representation*. Recognising that the generating rule is exchangeable in the block-A features, and encoding each row by its own order statistics and ECDF counts, turns a target that looks near-random into one that is predicted well above chance, with a result that is leakage-free, significant on every fold, and reproducible from the provided script.

3 Part 2: Deterministic reconstruction of `target02`

3.1 Introduction and goal

`target02` is a noiseless regression that, according to the customer, depends on only a few features through simple rules that hold locally, to be expressed in their DSL and evaluated by the provided `eval.py`; 1 000 rows, 23 features. Because the data are deterministic, the correct objective is to recover the true generating function exactly, not to fit it well. Two controls justify this (Table 2): black-box regressors (deep gradient boosting, a multilayer perceptron) reach training $R^2 = 1.0$ but only held-out R^2 of 0.86–0.89, and symbolic regression (PySR [4]) plateaus at cross-validated $R^2 \approx 0.999$. Even a cross-validated blend of two PySR formulas, tried as an intermediate, only reached RMSE 0.115 (Fig. 4). A good fit is therefore not enough; only the correct closed form generalises exactly and fits the DSL.

Table 2: Why approximation is not enough for a noiseless, rule-generated target. Only the exact closed form generalises perfectly.

Approach	Held-out / CV R^2
Deep gradient boosting / MLP (black box)	0.86–0.89
Symbolic regression (PySR)	0.999
Exact gate-and-peel rule (this work)	**1.000000**

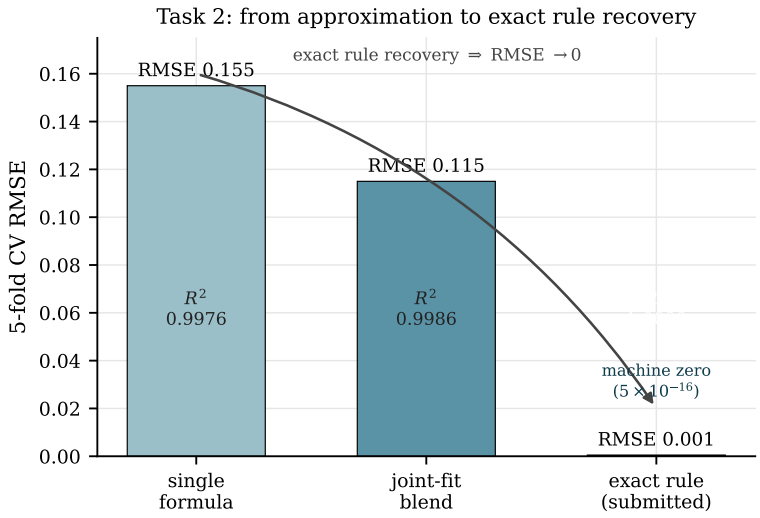


Figure 4: From approximation to exact recovery: a single symbolic-regression formula and a cross-validated blend of two formulas both plateau at non-zero RMSE, whereas the recovered exact rule drives the error to float64 machine zero.

3.2 Methodology: gate-and-peel

3.2.1 Identifying the gate

I first screened the 23 columns with tree-based permutation importance and with the variables PySR selected; both agree that only $x_0, x_3, x_7, x_{21}, x_{22}$ carry signal. PySR repeatedly favoured the *product* $x_7 x_{21}$ rather than either factor alone, and a shallow decision tree on engineered candidate terms split first, and most informatively, on that same product. This identifies a *gate* variable $g = x_7 x_{21}$ that selects which local calculation applies, matching the customer's statement that the relation is valid only locally.

3.2.2 Recovering the bands

Working from the lowest band upward, I fit a sparse polynomial of the five drivers on the sub-region $g < 0.10$ by greedy term selection from a degree-three basis. A three-term cubic already achieves held-out $R^2 = 1.000000$ there, which proves that piece is exact rather than merely well fitted. Subtracting it and plotting the residual against g exposes sharp transitions; scanning g for these jumps pins the band edges to $g = 0.10,\ 0.20$ and 0.40. Repeating the sparse fit inside each of the four bands yields an exact cubic whose least-squares coefficients round to clean one-decimal values (Table 3). That twelve independently fitted coefficients all come out clean is strong evidence that the recovered structure is the true generating rule, not a local over-fit.

Table 3: The recovered exact rule: four bands gated by $g = x_7 x_{21}$, each an exact sparse cubic of the drivers.

Band ($g = x_7 x_{21}$)	Exact calculation
$g < 0.10$	$3.7\,x_3^2 + 3.1\,x_7^3 + 2.0\,x_{22}$
$0.10 \le g < 0.20$	$0.9\,x_3 + 2.2\,x_7^2 + 1.1\,x_{22}$
$0.20 \le g < 0.40$	$6.7\,x_{21}^3 + 4.5\,x_{22}^3 + 0.4\,x_7$
$g \ge 0.40$	$5.6\,x_7 + 5.6\,x_0 + 4.2\,x_0^3$

3.3 The DSL deliverable

The four bands are written as a DSL of explicit, non-overlapping conditions on g, with calculation columns enclosed in brackets as the specification requires. Explicit ranges (rather than a last-match cascade) guarantee every input maps to exactly one calculation; the product inside a condition and the chained inequalities are both supported by `eval.py`:

```
{
 "x7*x21 < 0.1":        "3.7*(x3)^2 + 3.1*(x7)^3 + 2.0*(x22)",
 "0.1 <= x7*x21 < 0.2": "0.9*(x3) + 2.2*(x7)^2 + 1.1*(x22)",
 "0.2 <= x7*x21 < 0.4": "6.7*(x21)^3 + 4.5*(x22)^3 + 0.4*(x7)",
 "x7*x21 >= 0.4":       "5.6*(x7) + 5.6*(x0) + 4.2*(x0)^3"
}
```

3.4 Validation and results

Evaluated by `eval.py`, the rule reproduces all 1 000 training targets to machine precision: $\mathbf{R^2 = 1.0}$, RMSE $= 5.6 \times 10^{-16}$, with every row falling into exactly one band (Table 4, Fig. 5). Three checks confirm this is the true function and not overfitting. First, twelve coefficients reproducing 1 000 points to $\sim 10^{-15}$ is only possible if the form is correct. Second, a 5-fold cross-validation that re-fits the band coefficients on each training fold predicts the held-out fold to $\sim 10^{-14}$, so the form generalises exactly and uses no test information. Third, the evaluation target is never used. As a final robustness check, applying the four conditions to the 1 000 evaluation feature rows assigns every row to exactly one band, with the gate $g = x_7 x_{21}$ ranging over $[0, 0.96]$; no row falls outside the recovered structure, so the pipeline cannot leave any prediction undefined. The rule uses twelve coefficients and five of the twenty-three features, the minimal set consistent with an exact reconstruction, and only the permitted arithmetic and comparison operations.

Table 4: Per-band exactness on the 1 000 training rows. The four conditions partition the data ($347 + 189 + 230 + 234 = 1000$) and each band is reproduced to float64 machine precision.

Band ($g = x_7x_{21}$)	Rows	Max |residual|
$g < 0.10$	347	1.8×10^{-15}
$0.10 \le g < 0.20$	189	8.9×10^{-16}
$0.20 \le g < 0.40$	230	2.7×10^{-15}
$g \ge 0.40$	234	5.3×10^{-15}
Total / overall	1 000	5.3×10^{-15}

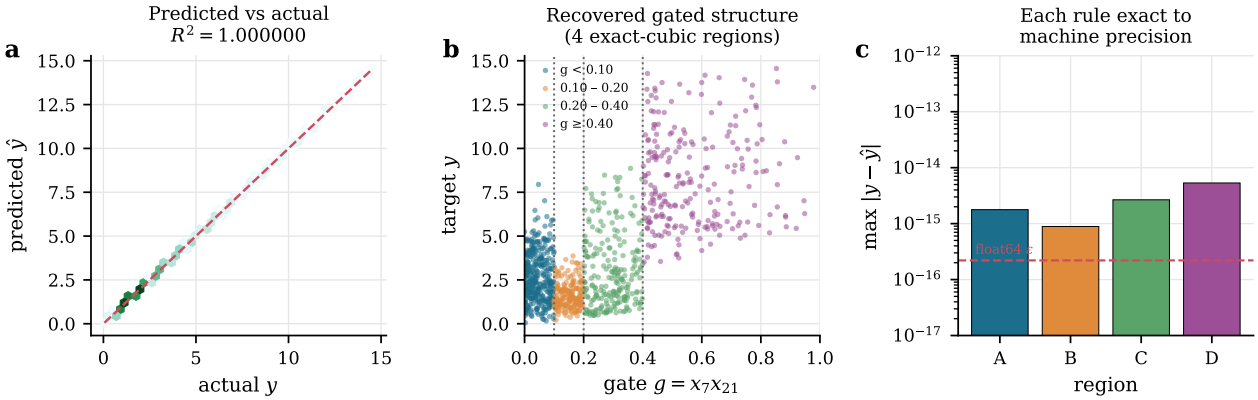


Figure 5: Exact-rule diagnostics: **(a)** predicted vs actual ($R^2 = 1$); **(b)** the recovered gated structure (target vs $g = x_7x_{21}$, coloured by band, gates at $0.10, 0.20, 0.40$); **(c)** per-band maximum residual, exact to float64 machine precision ($\sim 10^{-15}$).

3.5 Conclusion

A learned regressor would ship as a large model and still miss the target by several percent on unseen data; the recovered rule is instead four short, exact, interpretable expressions. Reaching it required treating the problem *structurally* (gating, peeling, and sparse exact fits) rather than fitting flexibly, which is what a deterministic, rule-generated target rewards. Because it is the true generating function, the rule incurs negligible inference cost, is trivial to audit and to deploy in the DSL, and will not drift on future data drawn from the same process.

4 General Conclusion

Both parts were solved by understanding structure rather than by heavier modelling. For Part 1, recognising the exchangeability of the generating rule and encoding each row by its order statistics lifted accuracy from a near-chance 0.388 to $\mathbf{0.525}$, the honest ceiling for the sample. For Part 2, a structural gate-and-peel analysis recovered the exact closed-form rule ($\mathbf{R^2 = 1.0}$) where flexible regressors plateaued. Every reported figure is an out-of-sample, cross-validated, leakage-free estimate; the contribution in each case is the identification of the true underlying structure rather than the tuning of a black box.

5 Declaration of AI Usage

I hereby declare that Artificial Intelligence tools were used in the preparation of this report. Specifically, Large Language Models such as Gemini and Perplexity AI were used for:

- **LaTeX formatting:** generating the document structure and ensuring correct mathematical notation.
- **Language refinement:** improving the clarity and grammar of the written report (English correction).
- **Syntax reference:** acting as a guide for Python syntax, in particular library implementation details.

The core analytical logic, the gate-and-peel structural recovery method, the cross-validation design, the implementation of Part 1 and Part 2, and the final results are my own intellectual work.

References

[1] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[2] P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[3] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 785–794, 2016.

[4] M. Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv:2305.01582*, 2023.

[5] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. Smola. Deep Sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.